University of Chittagong — Machine Learning (CSE 815) — Fully Solved Answer Script

Table of Contents

University of Chittagong

Department of Computer Science and Engineering

8th Semester B.Sc. Engg. Examination – 2021

Course: CSE 815 — Machine Learning — Fully Solved Answers

SECTION A

Question 1

1(a) What is "Machine Learning"? Give examples. — [2.75]

Definition: Machine Learning (ML) is a branch of Artificial Intelligence in which a system learns patterns and relationships directly from data and past experience, instead of being explicitly programmed with fixed rules. Formally (Tom Mitchell's definition): *"A computer program is said to learn from experience E with respect to some task T and performance measure P, if its performance at task T, as measured by P, improves with experience E."*

In simple words, we feed the algorithm example data (experience), it builds a mathematical model, and this model is then used to make predictions or decisions on new, unseen data.

Examples: 1. **Email spam filtering** – the system learns from thousands of labelled emails (spam/not-spam) and then classifies new incoming emails automatically. 2. **Netflix/Amazon recommendation systems** – learn a user's preferences from past viewing/purchase history to recommend new items. 3. **Handwritten digit recognition (e.g., postal code reading)** – the model learns from many labelled digit images and then recognises new handwritten digits. 4. **Medical diagnosis** – a model trained on patient records learns to predict disease risk from symptoms and test results. 5. **Self-driving cars** – learn to recognise lanes, pedestrians and obstacles from sensor data.

1(b) Describe different types of Machine Learning problems with necessary examples. — [3]

Machine learning problems are broadly categorised as follows:

Type	Description	Example
Supervised Learning	The model is trained on labelled data, i.e., each input has a known correct output. The goal is to learn a mapping from input → output. Sub-types: **Classification** (output is a discrete class label) and **Regression** (output is a continuous value).	Classification: spam vs. not-spam email; Regression: predicting house price from area/location
Unsupervised Learning	The model is trained on unlabelled data; it must discover hidden structure or groupings on its own. Sub-types: **Clustering** and **Dimensionality Reduction / Association**.	Clustering customers into segments based on purchase behaviour (K-Means); Market-basket association rule mining
Semi-Supervised Learning	Uses a small amount of labelled data together with a large amount of unlabelled data, since labelling is often expensive.	Web page classification, where only a few pages are manually labelled
Reinforcement Learning	An agent learns by interacting with an environment, taking actions and receiving rewards or penalties, in order to learn an optimal policy.	Game-playing agents (AlphaGo), robot navigation, self-driving car control

1(c) Describe different types of Machine learning methods with necessary examples. Make a comparison among different machine learning methods. — [3]

Different ML methods/algorithm families:

Method	Idea	Example Algorithm
Instance-based / Lazy learning	Stores training data and compares new data directly to it	k-Nearest Neighbour (k-NN)
Linear models	Fit a linear decision boundary/regression line	Linear Regression, Logistic Regression, Perceptron
Probabilistic (Generative) models	Learn the probability distribution of data	Naïve Bayes, Gaussian Mixture Models
Tree-based models	Split data recursively using feature tests	Decision Tree, Random Forest

Method	Idea	Example Algorithm
Kernel-based models	Map data to higher dimension to find a separating hyperplane	Support Vector Machine (SVM)
Neural Network / Deep Learning	Layers of interconnected neurons learn hierarchical features	Multilayer Perceptron, CNN, RNN
Ensemble methods	Combine several weak learners into a strong one	Bagging, Boosting (AdaBoost, XGBoost)

Comparison:

Criterion	k-NN	Decision Tree	Naïve Bayes	SVM	Neural Network
Training speed	Very fast (no real training)	Fast	Very fast	Slow (large data)	Slow, needs lots of data
Prediction speed	Slow (compares to all data)	Fast	Fast	Fast	Fast
Interpretability	Low	High (rules are readable)	Medium	Low	Very low (black box)
Handles non-linear data	Yes	Yes	Limited	Yes (with kernel)	Yes
Sensitive to noise/outliers	High	Medium	Low	Medium	Medium
Works well with small data	Yes	Yes	Yes	Yes	No (needs big data)

In short, simple models such as Naïve Bayes and Decision Trees are fast and interpretable but may underfit complex data, while SVMs and Neural Networks can model complex non-linear relationships at the cost of higher computational expense and lower interpretability.

Question 2

2(a) Define (i) FAR, (ii) FRR, (iii) Equal Error Rate (EER), (iv) Receiver Operating Characteristics (ROC), or Hit Ratio, (vii) Specificity, (viii) Sensitivity, (ix) Confusion Matrix. — [4.5]

(Numbering follows the original list; all requested terms are defined below.)

- **(i) FAR – False Acceptance Rate:** The proportion of impostor/negative instances that are incorrectly accepted (classified) as genuine/positive. FAR = FP / (FP + TN) *Example:*

In a fingerprint system, FAR is the fraction of unauthorized people wrongly granted access.

- **(ii) FRR – False Rejection Rate:** The proportion of genuine/positive instances that are incorrectly rejected (classified) as negative. FRR = FN / (FN + TP) *Example:* The fraction of authorized users wrongly denied access by a fingerprint scanner.
- **(iii) EER – Equal Error Rate:** The point (threshold) at which FAR equals FRR. A lower EER indicates a more accurate biometric/verification system, since it represents the best trade-off point between the two error types.
- **(iv) EER / Hit Ratio:** (see above) — **Hit Ratio** more generally refers to the fraction of correct predictions/detections out of all attempts, i.e., another name for recall/true positive rate in retrieval-type problems.
- **ROC – Receiver Operating Characteristic curve:** A plot of the True Positive Rate (Sensitivity) on the Y-axis against the False Positive Rate (1 – Specificity) on the X-axis at various classification thresholds. The **Area Under the Curve (AUC)** summarises overall classifier performance — AUC = 1 is a perfect classifier, AUC = 0.5 is random guessing.
- **F-score (F1-score):** The harmonic mean of Precision and Recall, used when we need a single balanced measure of a classifier especially with imbalanced classes: F1 = 2 × (Precision × Recall) / (Precision + Recall)
- **Hit Ratio / Recall:** Same as Sensitivity (see below) — proportion of actual positives that are correctly identified.
- **(vii) Specificity (True Negative Rate):** The proportion of actual negatives correctly identified as negative. Specificity = TN / (TN + FP)
- **(viii) Sensitivity (Recall / True Positive Rate):** The proportion of actual positives correctly identified as positive. Sensitivity = TP / (TP + FN)
- **(ix) Confusion Matrix:** A table that summarises the performance of a classification model by comparing predicted labels against actual labels.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

From this matrix, Accuracy, Precision, Recall, Specificity, F1-score, FAR and FRR can all be derived.

2(b) Discuss Overfitting vs Generalization with necessary example. — [2]

- **Overfitting** occurs when a model learns the training data too well — including its noise and random fluctuations — so that it performs excellently on training data but poorly on unseen/test data. It typically happens when the model is too complex relative to the amount of training data (e.g., a very deep decision tree that creates a separate leaf for

almost every training example, or a high-degree polynomial regression that passes through every training point).

- **Generalization** is the ability of a trained model to perform well on new, previously unseen data — this is the real goal of machine learning.

Example: Suppose we fit a polynomial curve to 10 data points. A straight line (degree-1) may under-fit and miss the trend; a degree-9 polynomial can pass exactly through all 10 points (zero training error) but oscillates wildly between points — it has memorised noise rather than the trend, and will predict poorly for new x-values. A moderate-degree polynomial (e.g., degree-2 or 3) usually generalizes best. Overfitting is commonly controlled using more training data, regularization (L1/L2), pruning (in decision trees), dropout (in neural nets), or cross-validation.

2(c) What is "cross-validation"? Explain 4-fold cross-validation methods. — [2.25]

Cross-validation is a technique used to evaluate how well a trained model will generalize to independent/unseen data. Instead of a single train/test split, the dataset is partitioned multiple times so that every observation gets a chance to be in both the training and the testing set, giving a more reliable estimate of model performance and helping detect overfitting.

4-Fold Cross-Validation: 1. The entire dataset is randomly divided into **4 equal-sized folds/subsets**: F1, F2, F3, F4. 2. In each of the 4 rounds, **one fold is kept aside as the test/validation set**, and the remaining **3 folds are used to train** the model. - Round 1: Train on F2, F3, F4 → Test on F1 - Round 2: Train on F1, F3, F4 → Test on F2 - Round 3: Train on F1, F2, F4 → Test on F3 - Round 4: Train on F1, F2, F3 → Test on F4 3. In each round, an accuracy/error score is recorded. 4. The **final performance estimate = average of the 4 scores** obtained across all rounds.

This ensures every data point is used exactly once for testing and three times for training, giving a robust, low-variance estimate of the model's true performance while making efficient use of limited data.

Question 3

3(a) Define: (i) input/hidden/output layers, (ii) weights, (iii) activation functions, and (iv) error. — [2]

- **(i) Input/Hidden/Output Layers:** In an artificial neural network, the **input layer** receives the raw feature values and passes them into the network without computation. **Hidden layer(s)** sit between input and output; they perform weighted summation and non-linear transformation to extract intermediate features/representations. The **output layer** produces the network's final prediction (e.g., a class score or a numeric value).

- **(ii) Weights:** Weights are the numeric parameters attached to each connection between neurons; they determine the strength/importance of the signal passed from

one neuron to the next. Learning in a neural network essentially means adjusting these weights (and biases) to minimize the error between predicted and target output.

- **(iii) Activation Functions:** A mathematical function applied to the weighted sum (net input) of a neuron to introduce non-linearity, allowing the network to learn complex, non-linear relationships. Common examples: Sigmoid, Tanh, ReLU, Hardlim (step function), Softmax.
- **(iv) Error:** The difference between the target (desired) output *t* and the actual output *a* produced by the network, e = t – a. This error signal drives the weight-update rule (e.g., in the Perceptron rule or Backpropagation) to gradually reduce prediction mistakes.

3(b) Define the Hebb rule. Analyze the application of Hebb rule using an autoassociative memory. — [3]

Hebb Rule: Proposed by Donald Hebb (1949): *"When one neuron repeatedly helps fire another, the connection (synapse/weight) between them is strengthened."* Mathematically, for input pattern **p** and target/output **t**, the weight update is:

$\Delta W = t \cdot p^T$ (and $\Delta b = t$, for the bias)

For a set of Q pattern pairs $\{p_1,t_1\},\{p_2,t_2\},\ldots,\{p_Q,t_Q\}$, the total weight matrix is obtained by summing the individual outer products:

$W = \Sigma$ (for q = 1 to Q) $t_q \cdot p_q^T = T \cdot P^T$

where **P** is the matrix whose columns are the input vectors and **T** is the matrix whose columns are the corresponding target vectors.

Application to Autoassociative Memory: In an autoassociative memory, each pattern is associated with **itself**, i.e., $t_q = p_q$. The Hebb rule then builds the weight matrix as:

$W = \Sigma$ (for q=1 to Q) $p_q \cdot p_q^T = P \cdot P^T$

This W is used to *recall* a stored pattern even from a noisy/incomplete version of it. When a new (possibly corrupted) pattern p is presented, the network computes:

$a = satlins(W \cdot p)$ (or hardlim/sign, depending on convention)

If the stored patterns are **orthogonal** ($p_i^T p_j = 0$ for $i \neq j$) and normalized, then presenting a stored pattern p_k exactly reproduces p_k, since:

$W \cdot p_k = \Sigma_q\, p_q\, (p_q^T p_k) = p_k\, (\|p_k\|^2) + \Sigma_{q \neq k}\, p_q\, (p_q^T p_k) = p_k \cdot \|p_k\|^2$ (cross terms vanish for orthogonal patterns)

So the network "recalls" the closest stored pattern — this is the basis of associative/content-addressable memory: given a partial or noisy key, the network reconstructs the complete, clean stored pattern. The more orthogonal the stored patterns are, the better the recall accuracy and the higher the memory capacity (fewer "crosstalk" errors between patterns).

3(c) Consider the three prototype patterns shown below (each is a 2×2 binary pixel pattern; black = +1, white = −1):

Note on reading the figure: the scanned patterns are read as follows (please re-check against your own copy of the figure — the method below applies unchanged even if a pixel differs):

- p_1 = (top row: black, black / bottom row: white, white) → $\mathbf{p_1 = [\,1\;1\;-1\;-1\,]^S}$
- p_2 = (top row: white, black / bottom row: black, white) → $\mathbf{p_2 = [\,-1\;1\;1\;-1\,]^S}$
- p_3 = (top row: black, white / bottom row: white, black) → $\mathbf{p_3 = [\,1\;-1\;-1\;1\,]^S}$
- p_4 (test pattern) = (top row: black, white / bottom row: white, black) → $\mathbf{p_4 = [\,1\;-1\;-1\;1\,]^S}$ (same as p_3, used to test recall)

(i) Use the Hebb rule to design a perceptron network (autoassociative memory) that will recognize these three patterns. — [2]

Since we want the network to reproduce each pattern from itself (autoassociative), apply the Hebb rule:

$W = p_1p_1^T + p_2p_2^T + p_3p_3^T$

With $p_1 = [1,1,-1,-1]^T$, $p_2 = [-1,1,1,-1]^T$, $p_3 = [1,-1,-1,1]^T$:

$p_1p_1^T$ =

```
 1  1 -1 -1
 1  1 -1 -1
-1 -1  1  1
-1 -1  1  1
```

$p_2p_2^T$ =

```
 1 -1 -1  1
-1  1  1 -1
-1  1  1 -1
 1 -1 -1  1
```

$p_3p_3^T$ =

```
 1 -1 -1  1
-1  1  1 -1
-1  1  1 -1
 1 -1 -1  1
```

Adding all three (element-wise):

W =

```
 3 -1 -3  1
-1  3  1 -3
-3  1  3 -1
 1 -3 -1  3
```

No bias term is required for a pure autoassociative Hebb network (b = 0). This W is the trained "perceptron-style" autoassociative memory.

(ii) Find the response of the network to the pattern p_4. Is the response correct? — [2]

Compute a = hardlims(W · p_4), where $p_4 = [1,-1,-1,1]^T$ and hardlims is the symmetric hard-limit (sign) function, applied element-wise.

W·p_4 :

Row1: 3(1) + (−1)(−1) + (−3)(−1) + 1(1) = 3+1+3+1 = 8 Row2: −1(1) + 3(−1) + 1(−1) + (−3)(1) = −1−3−1−3 = −8 Row3: −3(1) + 1(−1) + 3(−1) + (−1)(1) = −3−1−3−1 = −8 Row4: 1(1) + (−3)(−1) + (−1)(−1) + 3(1) = 1+3+1+3 = 8

$W \cdot p_4 = [8, -8, -8, 8]^T$

Applying hardlims (sign function): $a = [+1, -1, -1, +1]^T = p_4 (= p_3)$

Result: a = p_4, i.e., the network exactly reproduces the pattern that was presented. **Yes, the response is correct** — the autoassociative memory correctly recalls the stored pattern p_3 (which p_4 was set equal to), confirming that the Hebb-rule network works as a content-addressable memory for these prototype patterns.

Question 4

4(a) How would you apply a Naïve Bayes classifier to characteristics that are categorical? How about features that are numerical? — [2.75]

Categorical features: For a categorical attribute, the conditional probability P(feature = value | class) is estimated directly as a **frequency ratio**:

P(x_i = v | C_k) = (number of training instances in class C_k with x_i = v) / (total number of instances in class C_k)

(Laplace/Laplacian smoothing, i.e. adding 1 to the numerator and adding the number of possible categories to the denominator, is used to avoid zero probabilities when a value never appears with a class in the training data.)

Numerical (continuous) features: Since a continuous attribute can take infinitely many values, we cannot use simple frequency counts. Two common approaches: 1. **Gaussian Naïve Bayes:** Assume the feature values within each class follow a Normal (Gaussian) distribution. We estimate the mean (μ) and standard deviation (σ) of that feature for each class from the training data, and compute:

$P(x_i \mid C_k) = 1/(\sigma\sqrt{(2\pi)}) \cdot \exp(-(x_i - \mu)^2 / (2\sigma^2))$

2. **Discretization/Binning:** Convert the continuous attribute into discrete intervals/bins (e.g., Low/Medium/High) and then treat it like a categorical feature as above.

4(b) Does Naïve Bayes fit within the generative or discriminative classifier category? — [2]

Naïve Bayes is a **generative classifier**. It models the **joint probability distribution** $P(x, C) = P(C) \cdot P(x \mid C)$ — i.e., it learns how the data is generated for each class (the class prior and the class-conditional feature distributions) — and then uses Bayes' theorem to obtain the posterior $P(C \mid x)$ for classification:

$$P(C \mid x) \propto P(C) \cdot P(x \mid C)$$

This is in contrast to **discriminative classifiers** (e.g., Logistic Regression, SVM), which directly model the decision boundary/posterior $P(C \mid x)$ without modelling how the data was generated.

4(c) Consider the following data set for Naïve Bayesian classification and predict the class label for the test sample (Age = Low, Income = Fair, Credit_Rating = Low). — [4]

Given data:

Age	Income	Credit_Rating	Loan (Class)
High	Fair	Low	60,000
Low	Fair	High	70,000
Low	Excellent	High	80,000
High	Fair	High	65,000

Here the class attribute is **Loan**, and it has **4 distinct values** (60,000 / 70,000 / 80,000 / 65,000), each occurring exactly once in the (small) training set. Because every class has only a single training example, several conditional counts will be zero — so we apply **Laplacian (add-one) correction** as suggested in the question. Domain sizes: Age has 2 possible values {High, Low}; Income has 2 {Fair, Excellent}; Credit_Rating has 2 {Low, High}.

Prior probability for every class (1 instance out of 4 total): $P(C) = 1/4$ for each class.

For each class, using Laplace correction: $P(\text{attribute} = v \mid C) = (\text{count of } v \text{ in } C + 1) \,/\, (\text{count of } C + \text{number of possible values of that attribute}) = (\text{count} + 1)/(1 + 2)$

We need $P(\text{Age} = \text{Low} \mid C)$, $P(\text{Income} = \text{Fair} \mid C)$, $P(\text{Credit_Rating} = \text{Low} \mid C)$ for each class, then multiply by the prior.

Class (Loan)	Training row (Age, Income, Credit)	P(Age=Low|C)	P(Income=Fair|C)	P(Credit=Low|C)	P(C)×Product
60,000	(High, Fair, Low)	(0+1)/3 = 1/3	(1+1)/3 = 2/3	(1+1)/3 = 2/3	1/4 × 1/3 × 2/3 × 2/3 = **0.0370**
70,000	(Low, Fair, High)	(1+1)/3 = 2/3	(1+1)/3 = 2/3	(0+1)/3 = 1/3	1/4 × 2/3 × 2/3 × 1/3 = **0.0370**
80,000	(Low,	(1+1)/3 = 2/3	(0+1)/3 = 1/3	(0+1)/3 = 1/3	1/4 × 2/3 ×

Class (Loan)	Training row (Age, Income, Credit)	P(Age=Low|C)	P(Income=Fair|C)	P(Credit=Low|C)	P(C)×Product
	Excellent, High)				1/3 × 1/3 = **0.0185**
65,000	(High, Fair, High)	(0+1)/3 = 1/3	(1+1)/3 = 2/3	(0+1)/3 = 1/3	1/4 × 1/3 × 2/3 × 1/3 = **0.0185**

Conclusion: The two highest (and tied) posterior scores belong to classes **Loan = 60,000** and **Loan = 70,000** (score ≈ 0.0370 each), both clearly higher than the other two classes (≈0.0185). Since it is a tie, both are equally most probable outcomes under Naïve Bayes with Laplace correction for this test sample; in practice, a tie is usually broken by picking the class with the larger raw training frequency or the first one encountered — here we can report **Loan ≈ 60,000/70,000 (tied)** as the predicted class.

SECTION B

Question 5

5(a) Is it conceivable that during further K-Means rounds, the clustering of observations remains the same? — [2]

Yes. K-Means is guaranteed to **converge**: at each iteration the algorithm (1) re-assigns points to their nearest centroid and (2) recomputes centroids as the mean of the assigned points. This process strictly decreases (or keeps equal) the total within-cluster sum of squared distances (SSE) at every step, and since there is only a finite number of ways to partition a fixed data set into k groups, the algorithm must eventually reach a state where **no point changes cluster membership between one round and the next** — at that point the centroids stop moving and the clustering has **converged/stabilized**. So it is not only conceivable but expected that after enough rounds the clustering remains unchanged.

5(b) What do you understand by feature space? What is linearly separable and not separable? Explain with example. — [2.75]

Feature Space: The multi-dimensional space in which each axis represents one feature/attribute of the data, and each data instance is represented as a single point (vector) in this space. For example, if a dataset has 2 features (height, weight), each sample is a point in a 2-D feature space; with n features, each sample is a point in an n-dimensional feature space.

Linearly Separable: A dataset (with two classes) is **linearly separable** if there exists a straight line (in 2-D), a plane (in 3-D), or in general a hyperplane (in n-D) that can perfectly divide the data points of one class from the points of the other class. *Example:* Points of Class A clustered around (1,1),(2,1),(1,2) and Class B clustered around (8,8),(9,8),(8,9) — a single

straight line easily separates them → **linearly separable**. The classic example that works is the AND / OR logic gate truth tables.

Not Linearly Separable: If no single straight line/hyperplane can separate the two classes without misclassifying some points — the classes are said to be **not linearly separable**. *Example:* The classic **XOR problem** — points (0,0)→0, (1,1)→0 belong to one class, while (0,1)→1, (1,0)→1 belong to the other class. No single straight line can separate these two classes because they are arranged diagonally opposite each other; a non-linear boundary (or an additional hidden layer in a neural network) is required.

5(c) Divide the CGPA of the following students into two clusters using K-Means, initially designating A and C as the two cluster centers. Show only the final two clusters. — [3.75]

Data:

Student	A	B	C	D	E	F
CGPA	3.45	2.78	2.90	3.53	2.25	3.67

Initial centers: **Center-1 = A = 3.45**, **Center-2 = C = 2.90**

Iteration 1 — Assign each point to nearest center (1-D absolute distance):

Student	|x − 3.45|	|x − 2.90|	Assigned Cluster
A(3.45)	0.00	0.55	Cluster 1
B(2.78)	0.67	0.12	Cluster 2
C(2.90)	0.55	0.00	Cluster 2
D(3.53)	0.08	0.63	Cluster 1
E(2.25)	1.20	0.65	Cluster 2
F(3.67)	0.22	0.77	Cluster 1

Cluster 1 = {A, D, F} → new mean = (3.45+3.53+3.67)/3 = 10.65/3 = **3.55** Cluster 2 = {B, C, E} → new mean = (2.78+2.90+2.25)/3 = 7.93/3 = **2.6433**

Iteration 2 — Re-assign using updated centers (3.55 and 2.6433):

Student	|x − 3.55|	|x − 2.6433|	Assigned Cluster
A(3.45)	0.10	0.8067	Cluster 1
B(2.78)	0.77	0.1367	Cluster 2
C(2.90)	0.65	0.2567	Cluster 2
D(3.53)	0.02	0.8867	Cluster 1
E(2.25)	1.30	0.3933	Cluster 2
F(3.67)	0.12	1.0267	Cluster 1

The assignment is **identical to iteration 1** — no point changed cluster → the algorithm has **converged**.

Final Two Clusters:

- **Cluster 1 (higher CGPA):** {A = 3.45, D = 3.53, F = 3.67} — mean CGPA = **3.55**
- **Cluster 2 (lower CGPA):** {B = 2.78, C = 2.90, E = 2.25} — mean CGPA ≈ **2.64**

Question 6

6(a) Write short notes on Decision Tree. — [2]

A **Decision Tree** is a supervised-learning model represented as a tree structure, where: - Each **internal node** represents a test on a feature/attribute (e.g., "Is Age > 30?"), - Each **branch** represents the outcome of that test, - Each **leaf node** represents a class label (for classification) or a numeric value (for regression).

Construction: Built top-down (recursive partitioning) by choosing, at each step, the attribute that best splits the data into purer subsets. Common splitting criteria: **Information Gain / Entropy** (ID3, C4.5) or **Gini Index** (CART). The tree grows until stopping criteria are met (pure leaves, max depth, minimum samples, etc.).

Advantages: Easy to understand and visualize; requires little data preparation; handles both categorical and numerical data. **Disadvantages:** Prone to overfitting on noisy data (mitigated by pruning); small changes in data can produce a very different tree (high variance); biased toward attributes with many levels.

6(b) What is Edit Distance? Write down the Edit Distance algorithm. Find the Edit Distance between 'exercised' and 'executed'. — [2.25]

Edit Distance (Levenshtein Distance): The minimum number of single-character operations — **insertion**, **deletion**, or **substitution** — required to transform one string into another. It is widely used in spell-checking, DNA sequence alignment, and plagiarism detection.

Algorithm (Dynamic Programming): Given strings s1 (length m) and s2 (length n), build a (m+1)×(n+1) matrix D: 1. D[i][0] = i for i = 0..m (deleting i characters) 2. D[0][j] = j for j = 0..n (inserting j characters) 3. For i = 1..m, j = 1..n: - If s1[i] = s2[j]: D[i][j] = D[i−1][j−1] (no cost, characters match) - Else: D[i][j] = 1 + min(D[i−1][j] , D[i][j−1] , D[i−1][j−1]) (deletion, insertion, substitution) 4. The edit distance = D[m][n]

Computing Edit Distance between "exercised" (m=9) and "executed" (n=8):

		e	x	e	c	u	t	e	d
	0	1	2	3	4	5	6	7	8
e	1	0	1	2	3	4	5	6	7
x	2	1	0	1	2	3	4	5	6
e	3	2	1	0	1	2	3	4	5
r	4	3	2	1	1	2	3	4	5
c	5	4	3	2	1	2	3	4	5

		e	x	e	c	u	t	e	d
i	6	5	4	3	2	2	3	4	5
s	7	6	5	4	3	3	3	4	5
e	8	7	6	5	4	4	4	3	4
d	9	8	7	6	5	5	5	4	**3**

(Row labels = letters of "exercised", column labels = letters of "executed".)

Edit Distance("exercised", "executed") = 3

(One optimal alignment: exe**r**ci**s**e**d** → exe**cu**t**e**d — substitute r→c, insert u, substitute s/i-block appropriately; any sequence of 3 edit operations transforms one word into the other, and no fewer than 3 operations suffice.)

6(c) What is support vector? Why maximum margin in SVM technique? — [2]

Support Vectors: In a Support Vector Machine, the **support vectors** are the data points that lie closest to the separating hyperplane (decision boundary) — they are the critical points that directly determine the position and orientation of the boundary. If these points were removed or moved, the boundary would change; removing any other (non-support-vector) point would not affect the boundary at all.

Why Maximum Margin? SVM chooses the hyperplane that maximizes the **margin** — the distance between the hyperplane and the nearest data points of each class. A larger margin means: 1. The classifier is **less sensitive to small perturbations/noise** in the data (better robustness). 2. It gives the **best generalization** to unseen data, since a decision boundary that sits exactly midway between the two classes has the lowest risk of misclassifying new borderline points. 3. It has a strong theoretical basis (VC-dimension / structural risk minimization) showing that a maximum-margin classifier minimizes the upper bound on generalization error.

6(d) What is the "kernel trick"? How do you compute M in terms of W and b? — [2.5]

Kernel Trick: Many datasets are not linearly separable in their original feature space, but may become separable if mapped into a higher-dimensional space. Explicitly computing this mapping $\varphi(x)$ can be computationally very expensive (or even infinite-dimensional). The **kernel trick** avoids this cost: it uses a **kernel function** $K(x_i, x_j) = \varphi(x_i)\cdot\varphi(x_j)$ that computes the dot product of the mapped vectors **directly from the original inputs**, without ever explicitly performing the mapping φ. Common kernels: Linear, Polynomial $K(x,y) = (x^Ty + c)^d$, and RBF/Gaussian $K(x,y) = \exp(-\|x-y\|^2/2\sigma^2)$.

Computing the margin M in terms of W and b: For a linear SVM with decision boundary $W^Tx + b = 0$, the two supporting hyperplanes touching the nearest points of each class are $W^Tx + b = +1$ and $W^Tx + b = -1$. The perpendicular distance between these two parallel hyperplanes is the margin:

$M = 2 / \|W\|$

where $\|W\| = \sqrt{(w_1^2 + w_2^2 + \ldots + w_n^2)}$ is the Euclidean norm (magnitude) of the weight vector. SVM training maximizes M (equivalently minimizes $\|W\|^2/2$) subject to the constraint that every training point is correctly classified with at least unit distance from the boundary: $t_i(W^T x_i + b) \geq 1$ for all i.

Question 7

7(a) Demonstrate a proof of convergence for the perceptron learning rule, for a single neuron perceptron. — [4]

Perceptron Learning Rule: For input p, target t, output a = hardlim(Wp + b), error e = t − a, the update rule is:

$W(k+1) = W(k) + e \cdot p^T$ $b(k+1) = b(k) + e$

Convergence Theorem: If the training patterns are **linearly separable**, the perceptron learning rule is guaranteed to find a solution weight vector in a **finite number of steps**.

Proof sketch (standard proof, Rosenblatt/Hagan style):

Combine weight and bias into a single augmented weight vector **x** = [W; b] and augmented input **z** = [p; 1], so the update becomes simply $x(k+1) = x(k) + e \cdot z$ whenever a mistake occurs ($e \neq 0$).

Assume linear separability: there exists an optimal weight vector $\mathbf{x}^*$ such that $x^{*T} z > \delta > 0$ for every training vector belonging to the target-1 class (and $< -\delta$ for the target-0/−1 class), i.e., x^* classifies every pattern correctly with some positive margin δ.

Every time the perceptron makes a mistake on pattern z (with correct/target sign t = ±1), the update is $x(k+1) = x(k) + t \cdot z$.

1. **Growth of the "aligned component":** $x^{*T} x(k+1) = x^{*T} x(k) + t \cdot (x^{*T} z) \geq x^{*T} x(k) + \delta$ So after k mistakes: $x^{*T} x(k) \geq x^{*T} x(0) + k\delta$ — this grows **at least linearly** with k.

2. **Growth of the norm $\|x(k)\|$:** $\|x(k+1)\|^2 = \|x(k)\|^2 + 2t(x(k)^T z) + \|z\|^2 \leq \|x(k)\|^2 + \|z\|^2$ (since a mistake means $t \cdot (x(k)^T z) \leq 0$) Let $R^2 = \max\|z\|^2$ over all training patterns. Then after k mistakes: $\|x(k)\|^2 \leq \|x(0)\|^2 + kR^2$ — this grows **only as $\sqrt{k}$** (sub-linearly).

3. **Combining (Cauchy–Schwarz):** Since $x^{*T} x(k) \leq \|x^*\| \cdot \|x(k)\|$, we get: $x^{*T} x(0) + k\delta \leq \|x^*\| \cdot \sqrt{(\|x(0)\|^2 + kR^2)}$

   The left side grows linearly with k, while the right side grows only as $\sqrt{k}$. Since a linear function eventually exceeds any function growing as $\sqrt{k}$, this inequality can only hold for a **bounded number of mistakes $k \leq k_max$**. Once no more mistakes can occur, the algorithm has converged to a weight vector that correctly classifies all training patterns.

This shows that, for linearly separable data, the number of weight updates the perceptron rule makes is finite, hence the algorithm **converges** in a finite number of steps.

7(b) Solve the classification problem below using the perceptron rule; apply each input vector in order, repeating as needed, until the problem is solved. Draw the graph only after finding the solution. — [4.75]

Given: $p_1 = [2, 2]^T$, $t_1 = 0$ $p_2 = [1, -2]^T$, $t_2 = 1$ $p_3 = [-2, 2]^T$, $t_3 = 0$ $p_4 = [-1, 1]^T$, $t_4 = 1$ Initial: $W(0) = [0, 0]$, $b(0) = 0$. Rule: $a = \text{hardlim}(W \cdot p + b)$; $e = t - a$; $W \leftarrow W + e \cdot p^T$; $b \leftarrow b + e$. (hardlim(n)=1 if n≥0, else 0.)

Epoch 1:

Step	Input (p, t)	n = W·p + b	a	e = t−a	New W	New b
1	p_1=[2,2], t=0	0	1	−1	[0,0]+(−1)[2,2] = **[−2,−2]**	0−1 = **−1**
2	p_2=[1,−2], t=1	(−2)(1)+(−2)(−2)+(−1) = −2+4−1 = 1	1	0	[−2,−2] (no change)	−1 (no change)
3	p_3=[−2,2], t=0	(−2)(−2)+(−2)(2)+(−1) = 4−4−1 = −1	0	0	[−2,−2] (no change)	−1 (no change)
4	p_4=[−1,1], t=1	(−2)(−1)+(−2)(1)+(−1) = 2−2−1 = −1	0	1	[−2,−2]+1[−1,1] = **[−3,−1]**	−1+1 = **0**

End of Epoch 1: **W = [−3, −1], b = 0**

Epoch 2:

Step	Input (p, t)	n = W·p + b	a	e = t−a	New W	New b
1	p_1=[2,2], t=0	(−3)(2)+(−1)(2)+0 = −8	0	0	[−3,−1] (no change)	0
2	p_2=[1,−2], t=1	(−3)(1)+(−1)(−2)+0 = −3+2 = −1	0	1	[−3,−1]+1[1,−2] = **[−2,−3]**	0+1 = **1**
3	p_3=[−2,2], t=0	(−2)(−2)+(−3)(2)+1 = 4−6+1 = −1	0	0	[−2,−3] (no change)	1 (no change)
4	p_4=[−1,1], t=1	(−2)(−1)+(−3)(1)+1 = 2−3+1 = 0	1	0	[−2,−3] (no change)	1 (no change)

End of Epoch 2: **W = [−2, −3], b = 1** — no error remained in the last three steps.

Verification (Epoch 3, confirming convergence):

p	n = W·p+b	a	t	e
p_1=[2,2]	−2(2)−3(2)+1 = −9	0	0	0 ✓
p_2=[1,−2]	−2(1)−3(−2)+1 = 5	1	1	0 ✓
p_3=[−2,2]	−2(−2)−3(2)+1 = −1	0	0	0 ✓
p_4=[−1,1]	−2(−1)−3(1)+1 = 0	1	1	0 ✓

All four patterns are now classified correctly with **zero error** → the perceptron has **converged**.

Final solution: W = [−2, −3], b = 1

Decision boundary: W·p + b = 0 → $-2x_1 - 3x_2 + 1 = 0$ → **$2x_1 + 3x_2 = 1$**

Graph (decision boundary and points): - Line $2x_1 + 3x_2 = 1$, i.e. $x_2 = (1 - 2x_1)/3$, passes through (0.5, 0) and (−1, 1). - Points with t = 0 → p_1(2,2) and p_3(−2,2) lie on the side where $2x_1+3x_2 > 1$ (region classified as class 0… here output=0 when n<0, i.e. $2x_1+3x_2<1$ actually check: n = W·p+b = $-(2x_1+3x_2)+1$, so $n \geq 0 \iff 2x_1+3x_2 \leq 1$ → class 1. So points with $2x_1+3x_2 \leq 1$ are class 1 (p_2, p_4), and points with $2x_1+3x_2 > 1$ are class 0 (p_1, p_3). - p_1(2,2): 2(2)+3(2)=10>1 → class 0 ✓; p_3(−2,2): 2(−2)+3(2)=2>1 → class 0 ✓ - p_2(1,−2): 2(1)+3(−2)=−4≤1 → class 1 ✓; p_4(−1,1): 2(−1)+3(1)=1≤1 → class 1 ✓

So the straight line $2x_1+3x_2 = 1$ correctly separates {p_1, p_3} (class 0, above the line) from {p_2, p_4} (class 1, on/below the line).

Question 8 — Fruit Sorting Neural Network (Apple vs Orange)

Setup: Each fruit is represented by a 3-D vector p = [shape, texture, weight]^T, where each sensor outputs +1 or −1: - shape: +1 = round, −1 = elliptical - texture: +1 = smooth, −1 = rough - weight: +1 = weight > 1 lb, −1 = weight < 1 lb

(a) Prototype templates for orange (p_1) and apple (p_2). — [1]

A typical orange is round, rough-skinned, and light (< 1 lb); a typical apple is round, smooth-skinned, and heavier (> 1 lb):

Orange: $p_1 = [\,1, -1, -1\,]^S$ Apple: $p_2 = [\,1, 1, 1\,]^S$

(Target/class values: $t_1 = -1$ for orange, $t_2 = +1$ for apple, using bipolar targets.)

(b) Design a perceptron to recognize these patterns. — [1.5]

We need weights $W = [w_1\ w_2\ w_3]$ and bias b such that: $W \cdot p_1 + b < 0$ (→ orange, output −1) $W \cdot p_2 + b > 0$ (→ apple, output +1)

Since p_1 and p_2 agree on the shape component (both +1) but differ in texture and weight, we can ignore the shape input (weight it 0) and rely purely on texture & weight:

W = [0, 1, 1], b = 0

Check: for $p_1 = [1,-1,-1]$: $n = 0(1)+1(-1)+1(-1)+0 = -2 < 0$ → output −1 (orange) ✓ for $p_2 = [1,1,1]$: $n = 0(1)+1(1)+1(1)+0 = 2 > 0$ → output +1 (apple) ✓

This single-neuron perceptron (using symmetric hard-limit activation $a = hardlims(Wp+b)$) correctly separates the two fruit types with a simple decision plane $w_2+w_3=0$ in (texture, weight) space, ignoring shape entirely, since shape alone does not discriminate an apple from an orange in this simplified sensor model.

(c) Design a Hamming network to recognize these patterns. — [1.5]

A Hamming Network finds the stored prototype pattern that has the **smallest Hamming distance** (i.e., largest inner product) with the input.

Feed-forward layer: $W_1 = 0.5 \cdot P = 0.5\ [p_1^T; p_2^T] = 0.5\ [[1,-1,-1],[1,1,1]] =$

```
[ 0.5  -0.5  -0.5 ]
[ 0.5   0.5   0.5 ]
```

Bias: $b_1 = [R/2, R/2]^T = [1.5, 1.5]^T$ (R = number of elements = 3)

Output of feed-forward layer: $a^1 = W_1 \cdot p + b_1$ — each output neuron's value equals R minus the Hamming distance to that prototype (i.e., the number of matching bits).

Competitive (Maxnet) layer: a fully recurrent layer with self-excitation (weight = 1) and lateral inhibition (weight = $-\varepsilon$, a small negative value, $0 < \varepsilon < 1/(S-1)$ where S = number of neurons/classes) iteratively suppresses all outputs except the single largest one, which converges to a nonzero value while all others go to zero. The surviving neuron indicates the recognized class (orange or apple).

(d) Design a Hopfield network to recognize these patterns. — [1.5]

A Hopfield network stores patterns using the (auto-associative) Hebb rule and recalls them through recurrent dynamics. The weight matrix (with zero diagonal, since a neuron does not connect to itself):

$W = p_1 p_1^T + p_2 p_2^T - 2I$

$p_1 p_1^T =$

```
 1 -1 -1
-1  1  1
-1  1  1
```

$p_2 p_2^T$ =

```
1 1 1
1 1 1
1 1 1
```

Sum =

```
2 0 0
0 2 2
0 2 2
```

Subtract 2I (to zero the diagonal, since Q=2 patterns were summed on the diagonal):

W =

```
0 0 0
0 0 2
0 2 0
```

Recall dynamics: starting from an input pattern p, iteratively update each neuron a_i(k+1) = hardlims(Σ_j W_ij · a_j(k)) until the network settles into a stable state (an "energy minimum"), which should correspond to the closest stored prototype (p_1 or p_2).

(e) Test the operation of each network with several inputs; discuss advantages/disadvantages of each. — [3.25]

Test 1 — exact input p_1 = [1,−1,−1] (a perfect orange): - Perceptron: n = 0(1)+1(−1)+1(−1)=−2 <0 → output = −1 (orange) ✓ - Hamming: matches prototype 1 in all 3 bits (distance 0) and prototype 2 in 0 bits (distance 3) → neuron-1 wins → orange ✓ - Hopfield: W·p_1 = [0, −2, −2]^T → hardlims → [+1(unchanged if 0 treated as keep-previous), −1, −1] = p_1 → stable at p_1 → orange ✓

Test 2 — noisy input p = [1, −1, 1] (round, rough, but heavy — 1 bit different from orange): - Perceptron: n = 0(1)+1(−1)+1(1) = 0 → borderline/undecided (right on the boundary) — shows the perceptron only gives a hard binary decision and can be ambiguous exactly on the decision boundary. - Hamming: Hamming distance to p_1 = 1, to p_2 = 2 → neuron-1 (orange) still wins → correctly recovers the closer/intended prototype despite the noisy bit. - Hopfield: W·p = [0, −2, −2] → converges to [±1,−1,−1] = p_1 → correctly recalls orange, "cleaning up" the noisy bit.

Test 3 — exact input p_2 = [1,1,1] (a perfect apple): all three networks correctly output "apple" (analogous computation as Test 1).

Discussion — Advantages / Disadvantages:

Network	Advantages	Disadvantages
Perceptron	Very simple, fast, only 1 layer of weights; works well when classes are linearly separable	Gives only a hard binary decision; can fail/ be ambiguous exactly on the boundary; cannot handle

Network	Advantages	Disadvantages
		patterns that are not linearly separable; no explicit "closest match" notion
Hamming Network	Directly computes similarity (Hamming distance) to every stored prototype; naturally extends to any number of classes; robust to noisy/partial inputs since it picks the *closest* match, not just a binary decision	Requires a second (competitive/Maxnet) recurrent layer to pick the winner, which takes several iterations to converge; storage grows with the number of prototypes (one neuron per class)
Hopfield Network	Content-addressable / autoassociative — can reconstruct a complete, clean pattern even from a noisy or partial version, useful for pattern restoration; fully recurrent, elegant energy-minimization interpretation	Limited storage capacity (~0.15×N patterns for N neurons) — too many stored patterns cause incorrect recall (crosstalk); can converge to spurious ("false-memory") stable states that were never actually stored; recall requires iterative settling rather than a single feed-forward pass

Overall: For this simple 2-class fruit-sorting problem all three networks succeed, since orange and apple are easily separable by texture and weight; the Perceptron is the most efficient choice here (single-pass, single-layer). Hamming and Hopfield networks become more valuable as the number of prototype classes increases, or when the sensor readings are noisy/incomplete and pattern *reconstruction* (not just classification) is required.

End of solved answer script. All numerical computations (Naïve Bayes, K-Means, Edit Distance, Perceptron learning, Hebb rule) have been fully worked out step-by-step above.